Activity Selection Problem

-using Greedy Algorithm

Archisman De – 314

Manab Patra – 257

Souptik Bhattacharjee – 260

Interval Scheduling

Given a set of time intervals, select the maximum number of mutually non-overlapping intervals.



Unweighted Interval Scheduling

A scheduling problem where all activities have equal value, and the goal is to select the maximum number of non-overlapping activities. This is the problem commonly known as the Activity Selection Problem.

Weighted Interval Scheduling

A variation of the Activity Selection Problem where each activity has a different value or profit. The goal is to select a set of non-overlapping activities that maximizes the total value, rather than the number of activities.

Ways to solve

Approach	Time Complexity	Space	Optimal?	Remarks
Brute-force (all subsets)	$O(2^n)$	O(n)	Yes (searches all solutions)	Exponential, infeasible for large n
Backtracking/Branch & Bound	worst-case $O(2^n)$	O(n)	Yes (prunes search)	Exponential in worst case
Dynamic Programming (DP)	$O(n^2)$ (naive) or O(n log n) (with binary search)	O(n) to $O(n^2)$	Yes	Uses recurrence ; slower than greedy

Recap: Greedy Algorithm



Makes the best local choice at each step.



Builds the solution step-by-step without revisiting choices.



Aims to reach a globally optimal solution.

Why Greedy ??

DP: Considers multiple choices and stores the best results for subproblems, requiring additional memory.

Advantages of using Greedy Algorithm:

- Makes the proven optimal choice directly — select the earliest-finishing activity.
- Avoids maintaining a DP table and uses less extra storage.

- Achieves the same optimal result with simpler logic.

Example

Activity	Start Time	Finish Time
A1	3	5
A2	1	4
A3	8	11
A4	5	7
A5	0	6

After Sorting based on finish time

Activity	Start Time [S(t)]	Finish Time [F(t)]
A2	1	4
A1	3	5
A5	0	6
A4	5	7
A3	8	11

A2 A4 A3

Algorithm

```
GREEDY-ACTIVITY-SELECTOR(s,f):
    sort activities by finish time (so f[1] ≤ f[2] ≤ ... ≤ f[n])
    S = [ ];            // array of chosen activities
    last_finish = -∞;  // end time of last chosen activity
    j = 0;
    for i := 1 to n:
        if s[i] ≥ last_finish:
            S[j] =  A[i];     //A[] = array of activity names
            last_finish = f[i];
            j++;
    return S;
```

TIME AND SPACE COMPLEXITY FOR GREEDY APPROACH

Time Complexity

Sorting activities by finish time: O(n log n)

Selecting compatible activities: O(n)

Overall: O(n log n)

Space Complexity

Selection process uses O(1) auxiliary space

Overall: O(1) extra space, excluding the input and output storage

Key Point:

Sorting dominates the running time → O(n log n)

Proof of Optimality:-

- Assume an optimal solution starts with activity A.
- Greedy selects activity G, which finishes no later than A.
- Replace A with G in the optimal solution.
- Since G finishes earlier, it does not conflict with the remaining activities.
- Therefore, an optimal solution can start with G.
- Apply the same reasoning to the remaining activities.
- Hence, repeated greedy choices produce an optimal solution.

Application



Meeting room scheduling



Operation room scheduling



Broadcast / Communication scheduling

Advantage:-

- → Simple to understand and implement
- → Low extra memory requirement

Disadvantage:-

- → Can’t handle weighted activities
- → Assumes fixed intervals
- → Design for a single resource

Modern techniques:-



Weighted Interval Scheduling

→ Dynamic Programming



Multiple resources

→ Interval Partitioning



Changing intervals

→ Dynamic/online algorithms

References:-

- Kolen, A.W.J., Lenstra, J.K., Papadimitriou, C.H. and Spieksma, F.C.R. (2007), Interval scheduling: A survey. Naval Research Logistics, 54: 530-543. https://doi.org/10.1002/nav.20231
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- https://stackoverflow.com

Thank You !!